جزوه درجه یک سیستم عامل

فصل ۴ - بخش ۳: مدل‌های چندنخی

در بخش قبل فهمیدیم که "نخ" چیست. اما سوال اینجاست: سیستم‌عامل چطور این نخ‌ها را می‌بیند و مدیریت می‌کند؟

ما دو نوع نخ داریم که باید با هم کار کنند:

1. **نخ‌های سطح کاربر (User Threads):**
   - این نخ‌ها در **فضای کاربر (User Space)** زندگی می‌کنند.
   - مدیریت آن‌ها (ساختن، جابجایی بینشان) توسط یک "کتابخانه نخ" (مثل یک کتابخانه برنامه‌نویسی) انجام می‌شود.
   - **مزیت:** ساختن و مدیریت آن‌ها *بسیار سریع* است، چون نیازی به دخالت هسته (System Call) نیست.
   - **عیب:** هسته سیستم‌عامل اصلاً از وجود آن‌ها خبر ندارد!
1. **نخ‌های سطح هسته (Kernel Threads):**
   - این نخ‌ها مستقیماً توسط **هسته سیستم‌عامل (Kernel Space)** پشتیبانی و مدیریت می‌شوند.
   - هسته دقیقاً می‌داند که چند نخ هسته وجود دارد و خودش آن‌ها را زمان‌بندی می‌کند.
   - **مزیت:** هسته می‌تواند آن‌ها را به صورت *واقعاً موازی* روی هسته‌های مختلف CPU اجرا کند.
   - **عیب:** ساختن و مدیریت آن‌ها *کندتر* است، چون هر عملیات نیازمند یک System Call است.

حالا، "مدل چندنخی" به این می‌پردازد که این دو نوع نخ (کاربر و هسته) چطور به هم "نگاشت" (Map) می‌شوند. ما سه مدل اصلی داریم:

۴-۳-۱: مدل چند-به-یک (Many-to-One)

- **ایده:** "یک ارتش کارگر، با یک نماینده".
- تعداد **زیادی** نخ سطح کاربر (مثلاً ۱۰۰ نخ) همگی به **یک** نخ سطح هسته نگاشت می‌شوند.
- کل مدیریت نخ‌ها در سطح کاربر انجام می‌شود (بسیار سریع).

معایب (که باعث منسوخ شدن این مدل شد):

1. **فاجعه فراخوانی مسدودشونده:** اگر *فقط یکی* از آن ۱۰۰ نخ کاربر، یک فراخوانی سیستمی مسدودشونده (مثل خواندن از فایل) انجام دهد، "نخ هسته" (آن تنها نماینده) مسدود می‌شود. در نتیجه، **کل پردازه و ۹۹ نخ دیگر،** همگی مسدود می‌شوند!
2. **عدم موازی‌سازی:** چون در نهایت فقط "یک" نخ هسته وجود دارد، این پردازه هرگز نمی‌تواند به صورت موازی روی چند هسته CPU اجرا شود.

۴-۳-۲: مدل یک-به-یک (One-to-One)

- **ایده:** "هر کارگر، یک نماینده".
- هر نخ سطح کاربر **دقیقاً** به **یک** نخ سطح هسته نگاشت می‌شود.

مزایا (که باعث محبوبیت این مدل شد):

1. **همروندی واقعی:** اگر یک نخ مسدود شود، بقیه نخ‌ها به کار خود ادامه می‌دهند.
2. **موازی‌سازی واقعی:** هسته می‌تواند هر نخ را روی یک هسته CPU مجزا به صورت موازی اجرا کند.

عیب:

1. **سربار (Overhead):** ساختن هر نخ کاربر، نیازمند ساختن یک نخ *گران‌قیمت* در هسته است. اگر هزاران نخ بسازید، کارایی سیستم ممکن است افت کند.
- **محل استفاده:** این مدل، مدل رایج در **ویندوز و لینوکس** است.

۴-۳-۳: مدل چند-به-چند (Many-to-Many)

- **ایده:** "تعادل هوشمند".
- تعداد **زیادی** نخ سطح کاربر (مثلاً ۱۰۰ نخ) به تعداد **کمتری** نخ سطح هسته (مثلاً ۸ نخ) نگاشت می‌شوند.
- این مدل بهترین‌های هر دو جهان را می‌خواهد:
  - **سرعت:** ساختن نخ کاربر هنوز سریع است.
  - **همروندی:** اگر یک نخ کاربر مسدود شود، هسته می‌تواند نخ هسته آن را به یک نخ کاربر دیگر اختصاص دهد.
  - **موازی‌سازی:** می‌تواند روی ۸ هسته CPU به صورت موازی اجرا شود.

عیب:

1. **پیچیدگی وحشتناک:** پیاده‌سازی این مدل بسیار سخت و پیچیده است.
- **نکته:** به دلیل پیچیدگی، اکثر سیستم‌عامل‌های مدرن از همان مدل "یک-به-یک" استفاده می‌کنند، چون امروزه ساختن نخ هسته به اندازه کافی سریع شده است.

فصل ۴ - بخش ۴: کتابخانه‌های نخ (Thread Libraries)

این بخش به ما یاد می‌دهد که *چگونه* در عمل نخ بسازیم. "کتابخانه نخ" یک API (مجموعه توابع) است که ما برای ساختن و مدیریت نخ‌ها از آن استفاده می‌کنیم.

دو استراتژی کلی برای ساختن نخ:

1. **نخ‌سازی ناهمگام (Asynchronous):**
   - والد، فرزند را ایجاد می‌کند و بلافاصله به ادامه کارش می‌رسد. ("آتش کن و فراموش کن").
   - والد و فرزند مستقل از هم اجرا می‌شوند و معمولاً کاری به هم ندارند. (مثال: سرور وب).
1. **نخ‌سازی همگام (Synchronous):**
   - والد، فرزند (یا فرزندان) را ایجاد می‌کند و سپس **منتظر** می‌ماند تا کار تمام فرزندانش تمام شود.
   - بعد از اینکه همه فرزندان تمام شدند و به والد "ملحق" (Join) شدند، والد به ادامه کارش می‌رسد.
   - (مثال: برنامه‌ای که یک آرایه را برای پردازش موازی به ۴ نخ می‌دهد و منتظر جواب هر ۴ نخ می‌ماند).

ما سه کتابخانه اصلی را بررسی می‌کنیم:

۴-۴-۱: نخ‌های پوزیکس (Pthreads)

- استاندارد رایج در سیستم‌های **یونیکس، لینوکس و macOS**.
- داده‌های مشترک، متغیرهای **سراسری (Global)** هستند.

توابع کلیدی:

- `pthread_create(tid, ...)`: یک نخ جدید ایجاد می‌کند. ما باید آدرس تابعی (به نام `runner` در مثال) که می‌خواهیم نخ جدید آن را اجرا کند، به آن بدهیم.
- `pthread_join(tid, ...)`: (برای نخ‌سازی همگام). نخ والد را **مسدود** می‌کند تا زمانی که نخ فرزند (`tid`) کارش تمام شود و `exit` کند.
- `pthread_exit(0)`: نخ فرزند با این تابع کار خود را تمام می‌کند.

(مثال کد شکل ۴-۱۱ دقیقاً یک نخ‌سازی **همگام** را نشان می‌دهد).

۴-۴-۲: نخ‌های ویندوز (Windows Threads)

- مفاهیم شبیه پوزیکس هستند، فقط نام توابع فرق می‌کند.
- داده‌های مشترک، متغیرهای **سراسری (Global)** هستند.

توابع کلیدی:

- `CreateThread(...)`: یک نخ جدید ایجاد می‌کند و تابعی که باید اجرا شود را می‌گیرد (مثلاً `Summation`).
- `WaitForSingleObject(handle, ...)`: معادل دقیق `pthread_join` است. نخ والد را **مسدود** می‌کند تا کار نخ فرزند تمام شود.
- `WaitForMultipleObjects(...)`: (بسیار کاربردی) به نخ والد اجازه می‌دهد تا منتظر تمام شدن ***چندین*** نخ فرزند بماند.

۴-۴-۳: نخ‌های جاوا (Java Threads)

- جاوا نخ‌ها را به عنوان بخشی از زبان، در سطح JVM (ماشین مجازی جاوا) پیاده‌سازی می‌کند. JVM خودش این نخ‌ها را به نخ‌های سیستم‌عامل میزبان (مثلاً ویندوز یا پوزیکس) نگاشت می‌کند.
- **نکته مهم:** جاوا "متغیر سراسری" ندارد. داده‌های مشترک باید به عنوان "فیلد" (Field) در یک کلاس تعریف شوند که نخ‌ها به آن دسترسی دارند.

دو روش ساخت نخ:

1. ارث‌بری از کلاس `Thread` و بازنویسی (Override) متد `run()`.
2. پیاده‌سازی رابط (Interface) `Runnable` (و نوشتن کد در متد `run()`). **این روش رایج‌تر و بهتر است.**

شروع به کار نخ (بسیار مهم):

- شما **هرگز** متد `run()` را مستقیماً صدا نمی‌زنید!
- شما باید متد `start()` را روی شئ نخ صدا بزنید.
- `start()` دو کار می‌کند: ۱. یک نخ جدید در JVM می‌سازد. ۲. به آن نخ دستور می‌دهد که متد `run()` را اجرا کند.

انتظار کشیدن:

- `worker.join()`: معادل `pthread_join` در جاوا است. نخ والد را **مسدود** می‌کند تا نخ `worker` تمام شود.

چارچوب Executor جاوا (روش مدرن)

روش‌های بالا کمی قدیمی هستند. جاوا از نسخه ۱.۵ یک چارچوب بسیار قوی‌تر به نام `Executor` معرفی کرد.

ایده: "جدا کردن *تعریف* وظیفه از *اجرای* آن."

مشکل: `Runnable` (روش قدیمی) نمی‌تواند "نتیجه" برگرداند (متد `run` از نوع `void` است).

راه‌حل‌ها:

1. **رابط `Callable<V>`:** شبیه `Runnable` است، اما متد `call()` دارد که می‌تواند یک "نتیجه" (مثلاً یک `Integer`) برگرداند.
2. **شئ `Future<V>`:**
   - وقتی شما یک وظیفه `Callable` را به `Executor` می‌دهید (`submit()`)، او بلافاصله یک شئ `Future` به شما برمی‌گرداند.
   - این `Future` مثل یک "رسید" یا "قول" برای نتیجه است. نتیجه هنوز آماده نیست، اما شما قول آن را در دست دارید.
1. **گرفتن نتیجه:**
   - نخ والد به کارش ادامه می‌دهد.
   - هر زمان که به نتیجه نیاز داشت، متد `result.get()` را روی آن "قول" صدا می‌زند.
   - اگر نتیجه آماده بود، بلافاصله آن را می‌گیرد.
   - اگر هنوز آماده نبود، `get()` **مسدود** می‌شود تا زمانی که نخ فرزند کارش را تمام کند و نتیجه را برگرداند.

این روش بسیار قدرتمندتر است، چون به والد اجازه می‌دهد کارهای دیگری انجام دهد و فقط زمانی که ***واقعاً*** به نتیجه نیاز دارد، منتظر بماند.